

Algorithmique et programmation impérative

Chapitre 9 : Complexité : évaluer l'efficacité des algorithmes

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 9. Complexité : évaluer l'efficacité des algorithmes

- 1 Efficacité des algorithmes
- 2 Calculer le temps CPU des algorithmes
- 3 La notation grand- O et la notion de complexité dans le pire des cas
- 4 Evaluer la complexité des algorithmes itératifs
- 5 Evaluer la complexité des algorithmes récursifs
- 6 Analyse du comportement de nos algorithmes
- 7 Compléments à lire par soi-même
- 8 Glossaire

Référence

Le contenu de ce chapitre n'est pas couvert dans le livre *ThinkPython* de Downey.

En ce qui concerne la complexité, les références suivantes ont été utilisées :

- AHO, A. et ULLMAN, J., *Concepts fondamentaux de l'Informatique*, Dunod (1993)
- CORMEN, T. et LEISERSON, C., *Introduction to algorithms*, MIT Press (1991)

Efficacité des algorithmes

Efficacité des algorithmes

Etant donné un algorithme :

- comment évaluer son efficacité ?
- comment comparer l'efficacité de deux algorithmes différents résolvant le même problème ?

Pour cela, on peut :

- calculer le temps d'exécution utilisé par la machine pour effectuer un algorithme (on parle de temps **CPU**¹) : temps utilisé par le processeur pour réaliser une tâche particulière ;
- évaluer l'efficacité théorique d'un algorithme, la croissance du temps mis par un algorithme en fonction de la taille de ses entrées : on parle de la **complexité** de l'algorithme.

1. Central Processing Unit.

Efficacité des algorithmes

L'efficacité d'un algorithme peut dépendre de la **taille** des entrées (par ex. nombre d'éléments dans une liste, nombre d'équations dans un système à résoudre, etc.).

Pour une taille n donnée, l'efficacité d'un algorithme (par ex. un tri) peut également dépendre des **valeurs** données en entrée. On peut alors déterminer :

- le meilleur des cas (par ex. les valeurs sont déjà triées) ;
- le pire des cas (par ex. les valeurs sont triées par ordre décroissant).

Ainsi, on parlera de l'efficacité ou de la complexité d'un algorithme

- dans le pire des cas ;
- dans le meilleur des cas ;
- en moyenne (entrées aléatoires).

Calculer le temps CPU des algorithmes

Evaluer le temps CPU d'un algorithme

On ne calcule pas le temps d'exécution d'un algorithme avec un chronomètre !

En effet,

- le processeur n'est pas utilisé uniquement pour votre programme même si vous quittez toutes les autres applications. Il y a quantité de programmes qui tournent en tâches de fond (anti-virus, test de connexion internet, test pour savoir si un CD est dans le lecteur ou si un périphérique USB est branché, etc.)
- il y a aussi d'autres problèmes techniques difficiles à estimer : y-a-t il une utilisation de mémoire cache ? Comment cela marche-t-il ?

Evaluer le temps CPU d'un algorithme

On va plutôt évaluer le **temps CPU** : temps dédié par le processeur pour exécuter uniquement un (morceau) de code donné.

CPU = Central Processing Unit (processeur)

Le temps CPU est dépendant du processeur, du système d'exploitation, etc. On ne peut donc comparer des tests que s'ils sont réalisés sur une même machine (avec la même configuration).

Nous allons utiliser un module `umons_cpu` (disponible sur Moodle) qui permet d'estimer le temps CPU dépensé par une fonction à tester via une fonction `cpu_time`.

Utilisation du module `umons_cpu`

Pour illustrer le calcul du temps CPU, comparons deux algorithmes qui retournent le même résultat : le n ième nombre de Fibonacci.

Soit le module `fibonacci.py` contenant les deux fonctions suivantes :

```
def fib_rec(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib_rec(n-1) + fib_rec(n-2)

def fib_iter(n):
    f = [0, 1]
    for i in range(2, n+1):
        f.append(f[i-1] + f[i-2])
    return f[n]
```

Exemple d'utilisation de `umons_cpu.cpu_time`

```
>>> from umons_cpu import cpu_time
>>> from fibo import fib_iter, fib_rec
>>> fib_iter(30)
832040
>>> fib_rec(30)
832040
>>> cpu_time(fib_iter, 30)
0.009670206699956907
>>> cpu_time(fib_rec, 30)
481.5851666004164
>>> 481.58 / 0.00967
49801.447776628745
```

Syntaxe : `cpu_time(f, *args)` où `f` est la fonction à tester, suivie (d'un nombre variable) d'arguments qui sont les paramètres à donner à `f`. Retourne un temps CPU exprimé en millisecondes.

Exemple d'utilisation de `umons_cpu.cpu_time`

```
>>> from umons_cpu import cpu_time
>>> from fibo import fib_iter, fib_rec
>>> fib_iter(30)
832040
>>> fib_rec(30)
832040
>>> cpu_time(fib_iter, 30)
0.009670206699956907
>>> cpu_time(fib_rec, 30)
481.5851666004164
>>> 481.58 / 0.00967
49801.447776628745
```

Calculer le 30ième nombre de Fibonacci

- prend moins d'un centième de milliseconde pour la version itérative ;
- prend environ une demi seconde pour la version récursive ;
- est donc environ 50000 fois plus lent avec la version récursive !

Comparaison des temps CPU pour le calcul de Fibonacci

Temps affichés en msec

n:	iter:	rec:
5	0.003	0.004
10	0.005	0.033
15	0.006	0.353
20	0.007	3.946
25	0.008	43.773
30	0.010	487.291
100	0.026	----
300	0.075	----
500	0.129	----
700	0.190	----
900	0.245	----
1100	0.300	----
1300	0.359	----
1500	0.419	----
1700	0.479	----
1900	0.546	----

Pourquoi la version récursive est-elle beaucoup plus lente que la version itérative ? Une analyse de la **complexité** de ces algorithmes nous donnera la réponse.

Exemple d'utilisation de `umons_cpu.cpu_time`

Remarque : plus de détails concernant le module `umons_cpu` se trouvent à la fin du chapitre, dans la section à lire par soi même.

Vous y trouverez :

- des explications sur la manière dont les tests sont réalisés (il y a beaucoup de tests réalisés pour estimer un seul temps CPU et le nombre de tests est calibré automatiquement en fonction de la durée des premiers tests) ;
- des extraits de code montrant comment formater des tableaux de temps d'exécution comme celui présenté ci-avant.

La notation grand- O et la notion de complexité dans le pire des cas

Temps d'exécution

Pour analyser les temps d'exécution, nous allons définir une fonction $T(n)$ qui va représenter le nombre d'unités de temps pris par un algorithme sur une entrée de taille n , et ce, **dans le pire des cas**.

Pour simplifier les calculs, on va considérer que les instructions élémentaires prennent chacune **1 unité de temps** : assignation, opération arithmétique sur des nombres, etc.

Temps d'exécution

Exemple : soit la fonction suivante, comment exprimer $T(n)$ où n est le nombre d'éléments de la liste donnée en entrée.

```

1  def sum_all(liste):
2      res = 0
3      for item in liste:
4          res += item
5      return res

```

$T(n) = 2n + 2$ car

- l'instruction 2 est exécutée 1 fois
- l'instruction 3 est exécutée n fois (assignation de chaque élément)
- l'instruction 4 est exécutée n fois
- l'instruction 5 est exécutée 1 fois

C'est une approximation (on n'a pas tenu compte de l'appel à la fonction, du temps exact de chaque instruction élémentaire, du nombre exact d'instructions élémentaires cachées dans l'instr. 3, etc.) mais cela a peu d'importance : ce qui va nous intéresser est la croissance de $T(n)$, pas les constantes.

Temps d'exécution

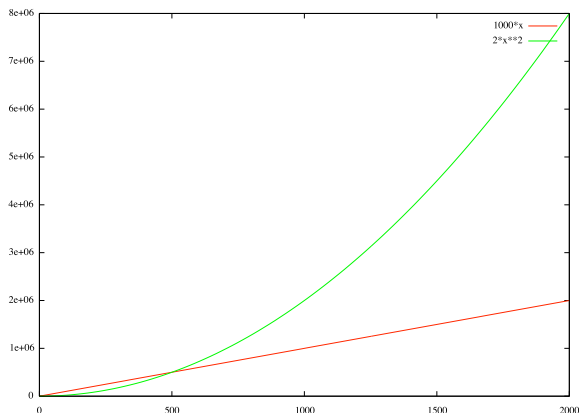
Ce qui nous intéresse c'est la **croissance** de $T(n)$, pas les constantes.

Imaginons que chaque instruction élémentaire soit réalisée en $1 \mu s$ (1 million d'instructions à la seconde) et que deux algorithmes A et B (pour le même problème) aient des temps $T_A(n) = 1000n$ et $T_B(n) = 2n^2$.

Temps d'exécution (en sec.) :

n	A	B
10	0.01	0.0002
100	0.1	0.02
1000	1.0	2.0
10000	10.0	200.0
100000	100.0	20000.0

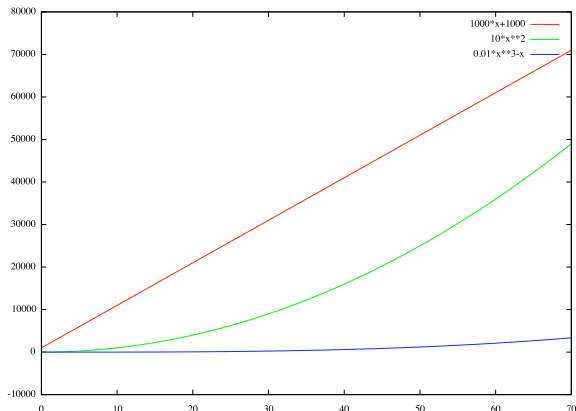
Temps d'exécution



Dès que $n > 500$, A est bien meilleur que B : la constante 1000 importe beaucoup moins que le fait que dans un cas on a une fonction linéaire, dans l'autre, elle est quadratique.

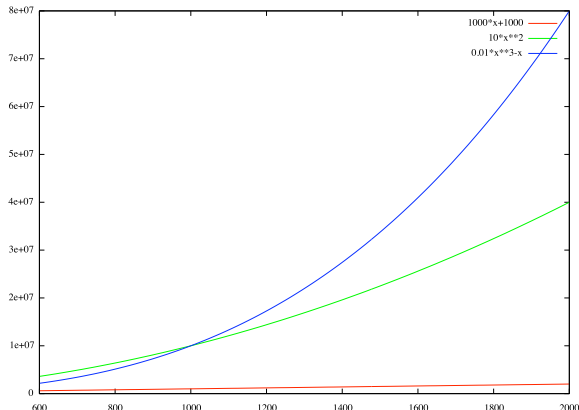
Temps d'exécution

Comparaison de $1000x + 1000$, $10x^2$ et $\frac{x^3}{100} - x$ sur $[0, 70]$



Temps d'exécution

Comparaison de $1000x + 1000$, $10x^2$ et $\frac{x^3}{100} - x$ sur $[600, 2000]$



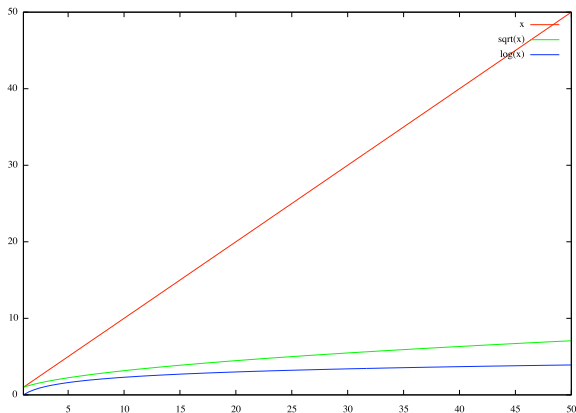
Temps d'exécution

Les fonctions de temps suivantes sont classées par croissance. Les termes fréquemment utilisés pour décrire certains temps sont mis entre parenthèses.

- $T(n) = \log n$ (temps logarithmique, la base importe peu)
- $T(n) = \sqrt{n}$
- $T(n) = n$ (temps linéaire)
- $T(n) = n \log n$
- $T(n) = n^2$ (temps quadratique)
- $T(n) = n^3$ (temps cubique)
- $T(n) = 2^n$ (temps exponentiel, la base importe peu)
- $T(n) = n!$

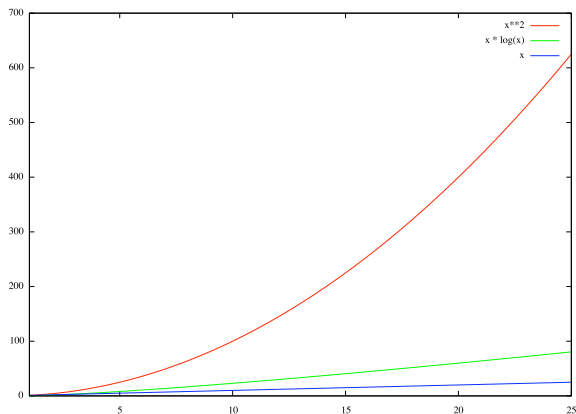
Temps d'exécution

Comparaison de $\log n$, $\sqrt{n}$ et n sur $[0, 50]$.



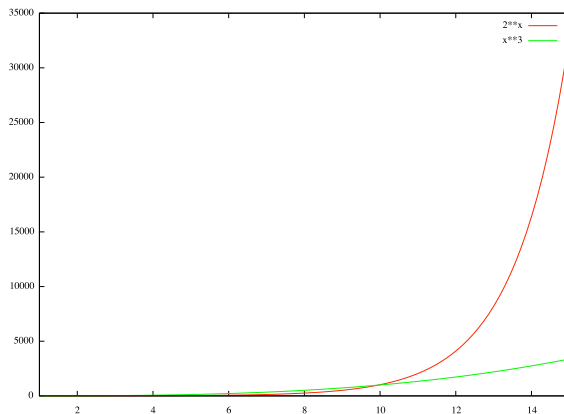
Temps d'exécution

Comparaison de n , $n \log n$ et n^2 sur $[0, 25]$.



Temps d'exécution

Comparaison de n^3 et 2^n sur $[0, 15]$.



Temps d'exécution

Comparaison de 2^n et $n!$ sur $[1, 20]$.

```
>>> import math
>>> for i in range(1, 21):
    print('%3d %7d %19d' % (i, 2**i, math.factorial(i)))
```

1	2	1
2	4	2
3	8	6
4	16	24
5	32	120
6	64	720
7	128	5040
8	256	40320
9	512	362880
10	1024	3628800
11	2048	39916800
12	4096	479001600
13	8192	6227020800
14	16384	87178291200
15	32768	1307674368000
16	65536	20922789888000
17	131072	355687428096000
18	262144	6402373705728000
19	524288	121645100408832000
20	1048576	2432902008176640000

Taille maximale du problème, pour un temps donné

Imaginons qu'une machine exécute 1 million d'instructions à la seconde et que nous sommes prêts à attendre 1 heure de temps CPU pour résoudre un problème. Quelle est la taille maximale du problème que l'algorithme A ($T_A(n) = 1000n$) peut résoudre ?

$$1000n \leq 60 \times 60 \times 10^6 = 36 \times 10^8,$$

donc $n \leq 36 \times 10^5$. L'algorithme A peut résoudre un problème de taille 3600000 en 1 heure.

Pour l'algorithme B ($T_B(n) = 2n^2$) :

$$2n^2 \leq 36 \times 10^8,$$

donc $n \leq \sqrt{18 \times 10^8} \simeq 42426$.

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour
$\ln n$	3×10^{434294}	8×10^{26057668}	$\simeq \infty$	$\simeq \infty$
$\sqrt{n}$	10^{12}	3.6×10^{15}	1.3×10^{19}	7.5×10^{21}
n	10^6	6×10^7	3.6×10^9	8.64×10^{10}
$n \ln n$	87847	3.9×10^6	1.8×10^8	3.9×10^9
n^2	1000	7745	60000	293938

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n							
$n!$							

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n	13						
$n!$							

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n	13	16	31	36	41		
$n!$							

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n	13	16	31	36	41	44	
$n!$							

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n	13	16	31	36	41	44	51
$n!$							

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n	13	16	31	36	41	44	51
$n!$	9						

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n	13	16	31	36	41	44	51
$n!$	9	11					

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n	13	16	31	36	41	44	51
$n!$	9	11	12				

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n	13	16	31	36	41	44	51
$n!$	9	11	12	13			

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n	13	16	31	36	41	44	51
$n!$	9	11	12	13	15		

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n	13	16	31	36	41	44	51
$n!$	9	11	12	13	15	16	

Taille maximale du problème, pour un temps donné

Supposons qu'un algorithme dépense un temps $f(n)$ pour résoudre un problème de taille n et que la machine exécute 1 million d'instructions à la seconde.

Taille maximale pour résoudre le problème en un temps donné :

$f(n)$	1 sec.	1 min.	1 heure	1 jour	1 mois	1 an	1 siècle
n^3	100	391	1532	4420	13886	31595	146652
2^n	13	16	31	36	41	44	51
$n!$	9	11	12	13	15	16	17

Notation grand- O

Soit $T(n)$ un temps d'exécution mesuré en fonction de la taille n du problème. Nous pouvons supposer que $T(n)$ est une fonction telle que :

- n est un entier positif ou nul ;
- $T(n) \geq 0 \quad \forall n$.

On va décrire le taux de croissance de ce type de fonction en utilisant une notation spécifique, la notation O (prononcer “grand- O ”). On parle aussi de la **complexité** (sous-entendu “dans le pire des cas”) de la fonction. Ceci permettra de classer les fonctions – et donc les temps d'exécution des algorithmes – selon leur type de croissance, indépendamment des constantes et autres détails techniques.

Notation grand- O

Les **fonctions de complexité** sont des fonctions

$$\mathbb{N} \rightarrow \mathbb{R}.$$

Intuition : Soit deux fonctions de complexité $f(n)$ et $g(n)$, on note

$$f(n) \in O(g(n)),$$

ou

$$f(n) = O(g(n)),$$

si f ne croît pas plus vite que g .

On dit alors que “ **f est en grand- O de g** ”.

Notation grand- O

“ f est en grand- O de g ”

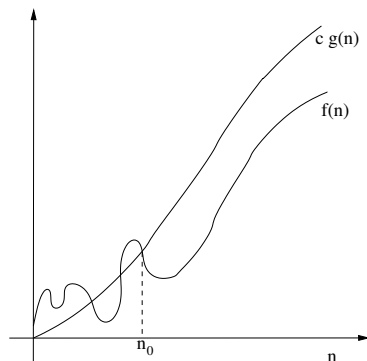
Intuition : f ne croît pas plus vite que g .

Définition formelle :

$$f(n) \in O(g(n)) \iff \exists n_0 \in \mathbb{N}, c \in \mathbb{R}^+ \text{ t.q. } \forall n \geq n_0 : f(n) \leq c g(n).$$

Notation grand- O

$$f(n) \in O(g(n)) \iff \exists n_0 \in \mathbb{N}, c \in \mathbb{R}^+ \text{ t.q. } \forall n \geq n_0 : f(n) \leq c g(n)$$



Vision asymptotique :

- $n_0 \geq 0$ permet d'éviter les effets de bord : ce n'est pas parce qu'un algorithme est plus rapide sur des petites valeurs, qu'il le sera encore asymptotiquement ;
- $c > 0$ exprime le fait que les constantes ne sont pas (très) importantes

Notation grand- O

$$f(n) \in O(g(n)) \iff \exists n_0 \in \mathbb{N}, c \in \mathbb{R}^+ \text{ t.q. } \forall n \geq n_0 : f(n) \leq c g(n)$$

Preuves : prouver qu'une fonction $f(n) \in O(g(n))$ revient à déterminer n_0 et c .

Exemple :

Soit $f(n) = 1000n$ et $g(n) = n$, alors $f(n) \in O(g(n))$. En effet, si $n_0 = 0$ et $c = 1000$, alors

$$f(n) = 1000n \leq 1000n = 1000g(n), \quad \forall n \geq 0.$$

D'autres exemples de telles preuves se trouvent dans la section à lire par soi même (fin du chapitre).

Simplification des expressions grand- O

La notation grand- O permet de simplifier les fonctions de complexité.

Lemme (les facteurs constants ne sont pas importants)

Pour toute valeur constante $d > 0$ et toute fonction de complexité $f(n)$, on a

$$f(n) \in O(d \cdot f(n)) \text{ et } d \cdot f(n) \in O(f(n))$$

Exemples : $1000n^2$ et $\frac{n^2}{1000} \in O(n^2)$.

Preuves pour ce lemme et les suivants : voir fin du chapitre

Simplification des expressions grand- O

Lemme (les termes d'ordre inférieurs sont négligeables)

Soit une fonction de complexité $f(n)$ polynomiale et dont le coefficient du terme de plus grand degré est positif, c-à-d,

$$f(n) := a_k n^k + a_{k-1} n^{k-1} + \dots + a_2 n^2 + a_1 n + a_0,$$

où $a_k > 0$. Alors,

$$f(n) \in O(n^k)$$

Exemple : $3n^5 + 10n^4 - 100n^3 + n + 1 \in O(n^5)$.

Simplification des expressions grand- O

On peut généraliser ce dernier lemme : si $g(n)$ croît moins vite que $h(n)$, alors

$$g(n) + h(n) \in O(h(n)).$$

(on peut négliger ce qui croît moins vite)

Exemple : toute exponentielle croît plus vite que tout polynôme :

$$2^n + n^3 \in O(2^n).$$

Simplification des expressions grand- O

Lemme (Transitivité des expressions grand- O)

Grand- O est une relation transitive, c-à-d que si $f \in O(g)$ et $g \in O(h)$, alors $f \in O(h)$.

Simplification des expressions grand- O

Pour exprimer la complexité d'une fonction $T(n)$ qui représente le temps d'exécution d'un algorithme, nous suivrons les deux règles suivantes :

- **Simplification.** On exprime $T(n)$ en appliquant les règles de simplification des expressions grand- O . Ainsi, si $T(n) = 2n^2 + 3$, on dira que l'algorithme est exécuté en un temps $O(n^2)$ (ou temps quadratique).
- **Proximité.** Il est clair que si $T(n) = 2n^2 + 3$, alors $T(n) \in O(n^3)$ ou même $O(2^n)$, mais cela n'exprime pas la croissance de $T(n)$. On dira donc que $T(n) \in O(n^2)$ (on choisit une fonction g dont la croissance est la plus faible possible et pour laquelle $f \in O(g)$.)

Evaluer la complexité des algorithmes itératifs

Evaluer la complexité des algorithmes

Nous allons appliquer la notation grand- O pour évaluer la complexité des algorithmes, en suivant quelques règles qui permettent de simplifier l'analyse et qui découlent directement des simplifications des expressions grand- O .

Au lieu d'évaluer un temps d'exécution $T(n)$ précisément, les simplifications des expressions grand- O vont être appliquées directement, en fonction des structures utilisées dans le programme à analyser (boucles, instructions conditionnelles, etc.)

Evaluer la complexité des algorithmes

- Une **instruction élémentaire** est une instruction qui réalise une action en un temps constant, quelle que soit la taille des entrées. Par exemple, les instructions suivantes sont des instructions élémentaires : affectation, opérations arithmétiques sur des entiers ou des réels, opérateur `[.]` d'accès à *un* élément d'une liste ou d'une chaîne, instruction `return`, etc.
- A l'inverse, toute instruction qui peut provoquer un temps d'exécution non constant n'est pas considérée comme une instruction élémentaire : appel d'une fonction ou d'une méthode (dont appels récursifs), boucles, instructions conditionnelles, certains opérateurs appliqués sur des objets de taille variable (comme l'opérateur "slice" sur les listes, son coût en temps peut ne pas être constant, dépendant de la taille de la tranche), etc.

Règle (Instruction élémentaire)

Par définition, une instruction élémentaire à un coût en temps $O(1)$ (temps constant).

Somme et produit

On “additionne” la complexité de morceaux de code en ne gardant que la complexité la plus grande.

Règle (Somme)

Soit deux parties A et B d'un programme, exécutées de façon séquentielle^a. Supposons $T_A(n) = O(f_A(n))$ et $T_B(n) = O(f_B(n))$. Alors

$$T_A(n) + T_B(n) = O(f_B(n) + f_A(n)) = O(\max(f_B(n), f_A(n))).$$

a. l'une après l'autre, la partie B n'est pas incluse dans le travail de la partie A et vice-versa.

Règle (Produit)

Soit deux fonctions f et g .

$$O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n)).$$

Somme et produit

Exemple

```
def swap(a, b):  
    temp = b  
    b = a  
    a = temp  
    return (a, b)
```

Le temps de cet algorithme est en $O(1)$ car sa complexité est $O(1) + O(1) + O(1) + O(1)$ (somme). De manière équivalente, il est en $O(1)$ car $4 \cdot O(1) = O(4) = O(1)$ (produit).

Appel de fonction

Règle (Appel de fonction ou méthode)

Le coût d'un appel à une fonction ou une méthode est équivalent au coût de l'exécution du corps de la fonction.

Remarque : on expliquera par après comment évaluer le coût d'une fonction récursive, mais la règle ci-dessus reste d'application.

Appel de fonction

Exemples

```
(a, b) = swap(a, b)
```

```
(a, b) = swap(a, b)
```

Le temps de cet algorithme (totalement inutile) est $O(1)$.

```
def f(n):
    res = g(n)
    res += h(n)
    return res
```

Imaginons que le temps d'exécution de $g(n)$ soit en $O(n)$ et celui de $h(n)$ soit en $O(n^2)$, alors le temps d'exécution de $f(n)$ est en $O(n) + O(n^2) + O(1) = O(n^2)$.

Complexité des opérations sur les listes

Observation : travailler (ajout / suppression) en **fin** de liste est plus efficace qu'au **début**

- obtenir la longueur de la liste ($n = \text{len}(t)$) : $O(1)$
- accéder à un élément d'un indice donné ($x = t[i]$) : $O(1)$
- ajouter un élément en fin de liste ($t.append(3)$) : $O(1)$
- supprimer et récupérer le dernier élément ($x = t.pop()$) : $O(1)$
- insérer un élément x à un indice i donné : $t.insert(i, x)$
 - ▶ pire des cas : au début de la liste ($t.insert(0, 8)$) : $O(n)$
 - ▶ meilleur des cas : en fin de liste ($t.insert(\text{len}(t), 8)$) : $O(1)$
- supprimer la première occurrence d'une valeur x ($t.remove(x)$) : $O(n)$

(d'après le site wiki python²)

2. <http://wiki.python.org/moin/TimeComplexity>

Complexité des opérations sur les listes – suite

- obtenir une tranche de taille k : $O(k)$
- copier une liste : $O(n)$ (par ex., la méthode `extend` copie la seconde liste à la fin de la première, donc $O(k)$ si la seconde liste à k éléments)
- concaténer : $O(n_1 + n_2)$
- multiplier (= copier) une liste k fois : $O(kn)$
- rechercher un élément dans la séquence (`x in s`) : $O(n)$
- trouver le min. ou le max. d'une séquence (`x = min(t)`) : $O(n)$
- trier une liste de taille n (`t.sort()`) : $O(n \log n)$
- l'appel à `range` est en $O(1)$ (mais est souvent utilisé dans une boucle, qui aura typiquement n éléments).

Complexité des opérations sur les listes

Exemples

```
t.append(3)  
print(3 in t)  
t2 = t * 2
```

Ce morceau de code a un temps linéaire :

$$O(1) + O(n) + O(2n) = O(n)$$

```
t2 = t * len(t)
```

Cette instruction a un temps quadratique : $O(n \cdot n) = O(n^2)$

Boucles `for`

Règle (Boucles `for`)

Le temps d'exécution d'une boucle `for` est équivalent à

complexité de la création de la séquence +
 $\#$ d'itérations (dans le pire des cas) $\times$ complexité du corps de la boucle

Remarques :

- la notation grand- O est utilisée dans ce chapitre pour calculer les temps d'exécution dans le pire des cas. Or, on peut sortir d'une boucle de différentes façons (`break`, `return`, condition d'arrêt), d'où le pire des cas sur le $\#$ d'itérations.
- $n \cdot O(1) = O(n)$ et pas $O(1)$: règle du produit
- pour multiplier une expression grand- O par quelque chose qui dépend de la taille du problème : $f(n) \cdot O(g(n)) = O(f(n) \cdot g(n))$ (règle du produit)

Boucles for

Exemple

```
1  def sum_all(liste):  
2      res = 0  
3      for item in liste:  
4          res += item  
5      return res
```

Le temps de cet algorithme est $O(n)$, où n est le nombre d'éléments de la liste.

Preuve.

La liste est déjà créée (car passée en paramètre), il n'y a donc pas de coût lié à son utilisation. Les instructions 2 et 5 sont en $O(1)$. La boucle est exécutée n fois. Le temps d'exécution du corps de la boucle (instr. 4) est en $O(1)$. Donc le temps de cet algorithme est $O(1) + n \cdot O(1) = O(n)$. □

Boucles for

Exemple

```

1  def sum_1_to_n(n):
2      res = 0
3      for i in range(1, n + 1):
4          res += i
5      return res

```

Le temps de cet algorithme est $O(n)$, où n est le nombre d'éléments de la liste.

Preuve.

La création de la séquence (fonction `range`) est en $O(1)$. Les instructions 2 et 5 sont en $O(1)$. La boucle est exécutée n fois. Le temps d'exécution du corps de la boucle (instr. 4) est en $O(1)$. Donc le temps de cet algorithme est $O(1) + O(n) + n \cdot O(1) = O(n)$. □

Boucles `while`

Règle (Boucles `while`)

Le temps d'exécution d'une boucle `while` est équivalent à

d'itérations (dans le pire des cas) $\times$ (complexité de l'évaluation de la condition + complexité du corps de la boucle)
+ complexité pour sortir de la boucle (condition)

Remarques :

- à chaque itération, la condition est évaluée, il faut donc en tenir compte.
- pour sortir de la boucle, il faut encore évaluer une fois la condition (voir un exemple où la condition est coûteuse dans la section à lire par soi même).

Boucles while

Exemple

```

1  def sum_1_to_n(n):
2      t = list(range(1, n + 1))
3      i = 0
4      res = 0
5      while i < n:
6          res += t[i]
7          i += 1
8      return res

```

Le temps de cet algorithme est $O(n)$, où n est le nombre d'éléments de la liste.

Preuve.

Le temps de l'instr. 2 est en $O(n)$. Les instructions 3, 4 et 8 sont en $O(1)$. La boucle est exécutée n fois et son corps est en $O(1)$. Le temps de l'évaluation de la condition est en $O(1)$. Donc le temps de cet algorithme est $O(n) + n \cdot (O(1) + O(1)) + O(1) = O(n)$. □

Algorithme : tri par insertion

Problème

Soit une séquence de n entiers ($n \geq 0$), comment trier (par ordre croissant) les éléments de la séquence ?

Idée du tri par insertion.

Imaginons que nous devons trier une main d'un jeu de cartes. On pose les cartes non triées sur la table (faces cachées). On prend une première carte en main : elle forme une "sous-main" triée. On prend une deuxième carte sur la table et on l'insère dans la main de telle sorte que les deux cartes en main soient triées. On répète le processus jusqu'à avoir inséré toutes les cartes posées sur la table.



Algorithme : tri par insertion

Idée du tri par insertion.

Pour appliquer cette idée à une séquence d'entiers, on représente la séquence comme suit : la partie grisée est la partie triée de la séquence.

3	7	2	6	5	1	4
3	7	2	6	5	1	4
2	3	7	6	5	1	4
2	3	6	7	5	1	4
2	3	5	6	7	1	4
1	2	3	5	6	7	4
1	2	3	4	5	6	7

Algorithme : tri par insertion

Idée du tri par insertion.

Pour appliquer cette idée à une séquence d'entiers, on représente la séquence comme suit : la partie grisée est la partie triée de la séquence.

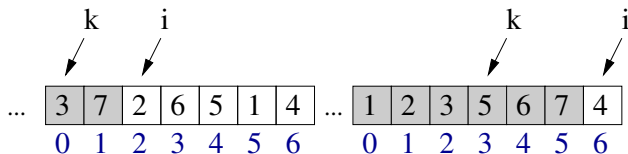
3	7	2	6	5	1	4
3	7	2	6	5	1	4
2	3	7	6	5	1	4
2	3	6	7	5	1	4
2	3	5	6	7	1	4
1	2	3	5	6	7	4
1	2	3	4	5	6	7

Deux boucles nécessaires :

- une boucle (indice i) pour trier chaque élément 1 à 1 (sauf le premier)
- une boucle interne à la première (indice j), pour décaler l'élément à trier vers la gauche jusqu'à sa place

Algorithme : tri par insertion

Exemple. Quand l'indice de la première boucle vaut i , les i premiers éléments sont triés (indices 0 à $i - 1$). Appelons *clef* la valeur de l'élément à trier. L'indice de la clef est i . Soit k la valeur de l'indice de la nouvelle position de la clef, après insertion : tous les éléments de la partie triée dont les valeurs sont plus grandes que la clef ont un indice compris entre k et $i - 1$. On va utiliser une deuxième boucle d'indice j pour décaler ces éléments vers la droite.



Algorithme : tri par insertion

Fonction en Python :

```

1  def insertion_sort(t):
2      n = len(t)
3      for i in range(1,n):
4          clef = t[i]
5          j = i - 1
6          while j >= 0 and t[j] > clef:
7              t[j+1] = t[j]
8              j = j - 1
9          t[j+1] = clef

>>> t = [3, 7, 2, 6, 5, 1, 4]
>>> insertion_sort(t)
>>> t
[1, 2, 3, 4, 5, 6, 7]
```

Remarque : à la ligne 9, le nouvel indice de la clef est $j + 1$ car on a décrémenté j lors de la dernière itération de la boucle while.

Analyse du tri par insertion

Quelle est la complexité dans le pire des cas du tri par insertion, en fonction du nombre n d'éléments à trier ?

```
1  def insertion_sort(t):
2      n = len(t)
3      for i in range(1,n):
4          clef = t[i]
5          j = i - 1
6          while j >= 0 and t[j] > clef:
7              t[j+1] = t[j]
8              j = j - 1
9          t[j+1] = clef
```

Analyse du tri par insertion

Quelle est la complexité dans le pire des cas du tri par insertion, en fonction du nombre n d'éléments à trier ?

```

1  def insertion_sort(t):
2      n = len(t)
3      for i in range(1,n):
4          clef = t[i]
5          j = i - 1
6          while j >= 0 and t[j] > clef:
7              t[j+1] = t[j]
8              j = j - 1
9          t[j+1] = clef

```

La complexité du tri par insertion est en temps $O(n^2)$.

Le tri par insertion est en $O(n^2)$ – preuve

```

1  def insertion_sort(t):
2      n = len(t)
3      for i in range(1,n):
4          clef = t[i]
5          j = i - 1
6          while j >= 0 and t[j] > clef:
7              t[j+1] = t[j]
8              j = j - 1
9          t[j+1] = clef

```

- l'instruction 2 est en $O(1)$
- considérons d'abord la boucle interne (`while`) : le corps de cette boucle est en $O(1)$ (instr. 7 et 8) et, dans le pire des cas, on sort de la boucle en ayant décalé tous les éléments, c-à-d quand le test $j \geq 0$ n'est plus vrai. Comme j est initialisé à $i - 1$, dans le pire des cas, on exécutera i fois la boucle (le nombre d'éléments entre l'indice 0 et l'indice $i - 1$ est i). Le temps de l'évaluation de la condition est en $O(1)$. La complexité totale de la boucle interne est donc $O(i)$ (dans le pire des cas $i = n - 1$ mais gardons cette expression en i pour le moment car il représente l'indice de la boucle extérieure)

Le tri par insertion est en $O(n^2)$ – preuve

```

1  def insertion_sort(t):
2      n = len(t)
3      for i in range(1,n):
4          clef = t[i]
5          j = i - 1
6          while j >= 0 and t[j] > clef:
7              t[j+1] = t[j]
8              j = j - 1
9          t[j+1] = clef

```

- considérons maintenant la boucle externe (for) : elle est exécutée exactement $n - 1$ fois. L'appel à range est en $O(1)$. Toutes les instructions du corps de la boucle sont en $O(1)$, sauf la boucle interne qui est en $O(i)$ (voir ci-avant). On peut calculer le temps total de la boucle comme suit grâce à la formule d'Euler :

$$O(n) + \sum_{i=0}^{n-1} O(i) = O(0 + 1 + \dots + n - 1 + n) = O\left(\sum_{i=1}^n i\right) = O\left(\frac{n^2 + n}{2}\right)$$

- On en conclut que le temps du tri par insertion est en $O(n^2)$ □

Et si la liste est déjà triée ? Voir fin du chapitre.

Instructions conditionnelles

Dans le cas d'une instruction conditionnelle, on ne garde que le plus grand temps d'exécution de toutes les *branches*³.

Règle (Instructions conditionnelles)

Soit une instruction conditionnelle avec k branches et ℓ conditions à évaluer, et soit $T_i(n)$ le temps d'exécution de la i ème branche. Si $T_i(n) \in O(f_i(n))$ pour $i = 1, \dots, k$, et si l'évaluation de la condition j est en $O(g_j(n))$, pour $j = 1, \dots, \ell$ alors

$$T(n) \in O\left(\max_i(f_i(n)) + \max_j(g_j(n))\right),$$

où $T(n)$ est le temps d'exécution de l'instruction conditionnelle complète.

3. Voir chap. 4.

Instructions conditionnelles

Exemple : attention aux opérations et appels de fonctions / méthodes cachés dans les conditions

```
if x in t:  
    print 'x est dans t'  
else:  
    print 'x pas dans t'
```

- les deux branches sont en $O(1)$
- le test du `if` est en $O(n)$ (où $n = \text{len}(t)$)
- la complexité est donc $O(n) + \max(O(1), O(1)) = O(n)$

Analyse du tri par sélection

Quelle est la complexité dans le pire des cas du tri par sélection, en fonction du nombre n d'éléments à trier ?

```
1  def selection_sort(t):
2      n = len(t)
3      for i in range(n-1):
4          small = i
5          for j in range(i+1, n):
6              if t[j] < t[small]:
7                  small = j
8          (t[i], t[small]) = (t[small], t[i])
```

Analyse du tri par sélection

Quelle est la complexité dans le pire des cas du tri par sélection, en fonction du nombre n d'éléments à trier ?

```
1  def selection_sort(t):  
2      n = len(t)  
3      for i in range(n-1):  
4          small = i  
5          for j in range(i+1, n):  
6              if t[j] < t[small]:  
7                  small = j  
8          (t[i], t[small]) = (t[small], t[i])
```

La complexité du tri par sélection est en temps $O(n^2)$.

Le tri par sélection est en $O(n^2)$ – preuve

```

1  def selection_sort(t):
2      n = len(t)
3      for i in range(n-1):
4          small = i
5          for j in range(i+1, n):
6              if t[j] < t[small]:
7                  small = j
8          (t[i], t[small]) = (t[small], t[i])

```

- l'instruction 2 est en temps constant ($O(1)$).
- la boucle interne (instr. 5 à 7) est en $O(n-i)$:
 - ▶ le corps de la boucle est en $O(1)$ car la condition du `if` et l'instr. 7 sont en $O(1)$
 - ▶ la construction de la séquence est en $O(n-i-1)$.
 - ▶ la boucle est exécutée exactement $n-i-1$ fois.
 - ▶ le temps total de cette boucle est donc

$$O(n-i-1) + O(n-i-1) \times O(1) = O(n-i)$$

Le tri par sélection est en $O(n^2)$ – preuve

```

1  def selection_sort(t):
2      n = len(t)
3      for i in range(n-1):
4          small = i
5          for j in range(i+1, n):
6              if t[j] < t[small]:
7                  small = j
8          (t[i], t[small]) = (t[small], t[i])

```

● la boucle externe (instr. 3 à 8) est en $O(n^2)$:

- ▶ la construction de la séquence est en $O(n-1)$.
- ▶ la boucle est exécutée exactement $n-1$ fois.
- ▶ le corps de la boucle est en $O(n-i)$ (coût de la boucle interne) car les instructions 4 et 8 sont en temps constant.
- ▶ le temps total de cette boucle est donc

$$O(n-1) + \sum_{i=0}^{n-2} O(n-i) = O(n-1) + O(n + (n-1) + \dots + 2) = O(n^2),$$

par la formule d'Euler (cf. preuve du tri par insertion) $\square$

Et si la liste est déjà triée ? Voir fin du chapitre.

Evaluer la complexité des algorithmes récurrents

Evaluer la complexité des algorithmes récursifs

Pour analyser le temps d'exécution $T(n)$ d'une fonction **récursive** f :

- on détermine une formule **inductive** pour $T(n)$:
 - ▶ *Base*. Que vaut $T(n)$ quand il n'y a pas d'appel récursif ?
 - ▶ *Gén.* Que vaut $T(n)$ quand il y a des appels récursifs ? On estime le temps $T(n)$ par rapport au temps $T(k)$ des appels récursifs où k est la taille appropriée des arguments (par exemple, $T(n-1)$).
- on évalue **deux fois** le temps du corps de f comme précédemment, mais en gardant les termes $T(k)$ sous la forme d'inconnues.
 - ▶ quand on tombe dans le(s) cas de base
 - ▶ dans le cas général
- Dans les expressions obtenues, on remplace les termes grand- O comme $O(h(n))$ par $c \cdot h(n)$ où c est une constante particulière.
- On en déduit une forme **non-inductive** de $T(n)$

Evaluer la complexité des algorithmes récurrents

Exemple

```

1  def sum_rec(n):
2      if n == 1:
3          return 1
4      else:
5          return sum_rec(n-1) + n

```

- pour la base de la définition inductive de $T(n)$, on choisit $n = 1$. Dans ce cas, la complexité de $T(1)$ est en $O(1)$ (instr. 2 et 3).
- si $n > 1$, seules les instr. 2 et 4 et 5 sont exécutées. Les instructions 2 et 4 sont en $O(1)$ et le temps d'exécution de l'instr. 5 est $T(n-1) + O(1)$.
- la forme **inductive** de $T(n)$ est donc :

$$\begin{aligned}
 T(1) &= O(1), \\
 T(n) &= T(n-1) + O(1), \quad \forall n > 1.
 \end{aligned}$$

Evaluer la complexité des algorithmes récursifs

- on remplace les expressions $O(h(n))$ par $c \cdot h(n)$ (une constante pour chaque expression) :

$$\begin{aligned} T(1) &= a, \\ T(n) &= T(n-1) + b, \quad \forall n > 1. \end{aligned}$$

- on exprime $T(n)$ en fonction de n et des constantes. Pour les premières valeurs on a :

$$\begin{aligned} T(1) &= a, \\ T(2) &= T(1) + b = a + b, \\ T(3) &= T(2) + b = a + 2b, \\ T(4) &= T(3) + b = a + 3b. \end{aligned}$$

- on déduit que $T(n) = a + (n-1)b, \forall n \geq 1$ (forme **non-inductive**)
- on conclut que $T(n) \in O(n)$ puisque a et b sont des constantes

Evaluer la complexité des algorithmes récursifs

Remarque : nous nous sommes basés sur des "expérimentations" pour déterminer la forme non-inductive

$$T(n) = a + (n - 1)b, \forall n \geq 1.$$

Ce n'est donc pas une preuve. Mais il est simple de prouver par récurrence (exercice) que cette formule implique la forme inductive

$$\begin{aligned} T(1) &= a, \\ T(n) &= T(n-1) + b, \quad \forall n > 1. \end{aligned}$$

Algorithme : tri par fusion

Problème

Soit une séquence de n entiers ($n \geq 0$), comment trier (par ordre croissant) les éléments de la séquence ?

Idée du tri par fusion.

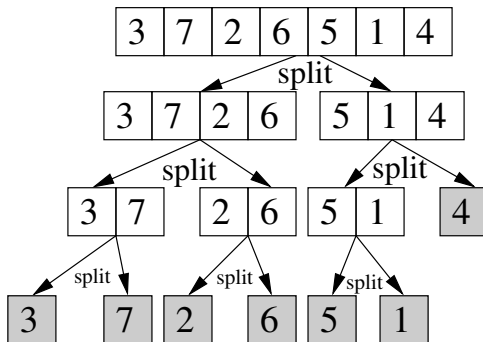
Imaginons que nous devons trier une main d'un jeu de cartes et que Bob soit en notre compagnie. On divise le paquet de cartes en 2 parties (à peu près) égales. On donne la première partie à Bob, qui s'occupe de la trier. On lui donne alors la seconde partie pour qu'il la trie également. Ensuite, à partir des deux paquets de cartes triées, on reconstitue un seul paquet de cartes trié en fusionnant les deux paquets triés.



Algorithme : tri par fusion

Idée du tri par fusion.

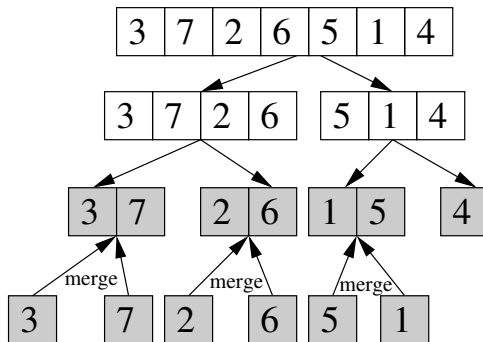
Appliquons cette idée à une séquence d'entiers : on divise (split) et on fusionne deux parties triées (merge). Une séquence à 1 élément est triée. Les séquences triées sont grisées.



Algorithme : tri par fusion

Idée du tri par fusion.

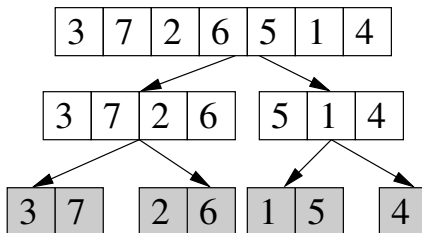
Appliquons cette idée à une séquence d'entiers : on divise (split) et on fusionne deux parties triées (merge). Une séquence à 1 élément est triée. Les séquences triées sont grisées.



Algorithme : tri par fusion

Idée du tri par fusion.

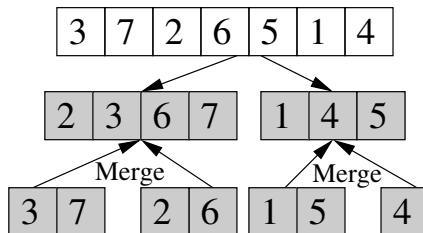
Appliquons cette idée à une séquence d'entiers : on divise (split) et on fusionne deux parties triées (merge). Une séquence à 1 élément est triée. Les séquences triées sont grisées.



Algorithme : tri par fusion

Idée du tri par fusion.

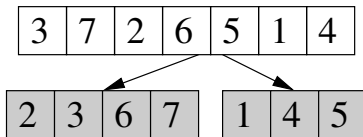
Appliquons cette idée à une séquence d'entiers : on divise (split) et on fusionne deux parties triées (merge). Une séquence à 1 élément est triée. Les séquences triées sont grisées.



Algorithme : tri par fusion

Idée du tri par fusion.

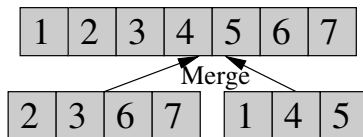
Appliquons cette idée à une séquence d'entiers : on divise (split) et on fusionne deux parties triées (merge). Une séquence à 1 élément est triée. Les séquences triées sont grisées.



Algorithme : tri par fusion

Idée du tri par fusion.

Appliquons cette idée à une séquence d'entiers : on divise (split) et on fusionne deux parties triées (merge). Une séquence à 1 élément est triée. Les séquences triées sont grisées.



Algorithme : tri par fusion

Idée du tri par fusion.

Appliquons cette idée à une séquence d'entiers : on divise (split) et on fusionne deux parties triées (merge). Une séquence à 1 élément est triée. Les séquences triées sont grisées.

1	2	3	4	5	6	7
---	---	---	---	---	---	---

Algorithme : tri par fusion

Idée du tri par fusion.

On va écrire

- une fonction `split` pour séparer une liste (approx.) en 2 ;
- une fonction `merge` qui fusionne deux listes triées ;
- une fonction récursive `merge_sort`
 - ▶ **Base** : une liste vide ou à un seul élément est une liste triée ;
 - ▶ **Gén.** : on divise la liste en 2, puis on trie récursivement les 2 parties, puis on fusionne les deux parties triées.
- contrairement aux tris par sélection et insertion qui ne modifient que les valeurs à l'intérieur de la liste⁴, ici nous devons travailler avec des nouvelles sous-listes. Pour éviter les problèmes (cf. Chap. 7), les fonctions retourneront les listes.

```
insertion_sort(t)  # la liste est modifiée par la fonction
t = merge_sort(t) # on retourne la liste triée
```

4. On parle de fonctions réalisant le travail “in place”.

Algorithme : tri par fusion

Fonction `split` en Python :

```

1  def split(t):
2      """ precondition: len(t) >= 2 """
3      mid = len(t) // 2
4      t1 = t[:mid]
5      t2 = t[mid:]
6      return (t1, t2)

>>> split([1,2])
([1], [2])
>>> split([1,2,3])
([1], [2, 3])
>>> split(list(range(10)))
([0, 1, 2, 3, 4], [5, 6, 7, 8, 9])

```

Algorithme : tri par fusion

Idée de l'algorithme merge.

On va écrire une fonction récursive `merge` qui fusionne deux séquences A_1 et A_2 triées :

- **Base** : si A_1 est vide, alors A_2 est la liste fusionnée ; si A_2 est vide, alors A_1 est la liste fusionnée⁵
- **Gén.** : soit x le dernier élément de A_1 et y le dernier élément de A_2 . Si $x > y$, alors x doit se trouver en dernier dans la liste fusionnée⁶. La liste fusionnée est $merge(A'_1, A_2)$ avec x ajouté à la fin et où A'_1 est A_1 pour laquelle on a retiré le dernier élément. Le cas $x \leq y$ est géré de manière symétrique.

5. Implique : si les deux listes sont vides, alors la liste fusionnée est vide.

6. On travaille en fin de liste plutôt qu'en début car on verra que c'est plus efficace un peu plus tard.

Algorithme : tri par fusion

Fonction `merge` en Python :

```

1  def merge(t1, t2):
2      if len(t1) == 0:
3          return t2
4      elif len(t2) == 0:
5          return t1
6      elif t1[-1] > t2[-1]:
7          last = t1.pop()
8          new = merge(t1, t2)
9          new.append(last)
10         return new
11     else:
12         last = t2.pop()
13         new = merge(t1, t2)
14         new.append(last)
15         return new

>>> merge([], [])
[]
>>> merge([], [1])
[1]
>>> merge([1, 2, 7, 8], [3, 4, 9])
[1, 2, 3, 4, 7, 8, 9]

```

Algorithme : tri par fusion

Fonction `merge_sort` en Python :

```

1  def merge_sort(t):
2      n = len(t)
3      if n > 1:
4          (t1, t2) = split(t)
5          t1 = merge_sort(t1)
6          t2 = merge_sort(t2)
7          return merge(t1, t2)
8      else:
9          return t

>>> merge_sort(t)
[1, 2, 3, 4, 5, 6, 7]
>>> t
[7, 4, 2, 1, 6, 5, 3]
>>> t = merge_sort(t)
>>> t
[1, 2, 3, 4, 5, 6, 7]

```

Analyse du tri par fusion

Quelle est la complexité dans le pire des cas du tri par fusion, en fonction du nombre n d'éléments à trier ?

```

1  def merge_sort(t):
2      n = len(t)
3      if n > 1:
4          (t1, t2) = split(t)
5          t1 = merge_sort(t1)
6          t2 = merge_sort(t2)
7          return merge(t1, t2)
8      else:
9          return t
10
11 def split(t):
12     """ precondition: len(t) >= 2 """
13     mid = len(t) // 2
14     t1 = t[:mid]
15     t2 = t[mid:]
16     return (t1, t2)
17
18 def merge(t1, t2):
19     if len(t1) == 0:
20         return t2
21     elif len(t2) == 0:
22         return t1
23     elif t1[-1] > t2[-1]:
24         last = t1.pop()
25         new = merge(t1, t2)
26         new.append(last)
27         return new
28     else:
29         last = t2.pop()
30         new = merge(t1, t2)
31         new.append(last)
32         return new

```

Analyse du tri par fusion

Quelle est la complexité dans le pire des cas du tri par fusion, en fonction du nombre n d'éléments à trier ?

```

1  def merge_sort(t):
2      n = len(t)
3      if n > 1:
4          (t1, t2) = split(t)
5          t1 = merge_sort(t1)
6          t2 = merge_sort(t2)
7          return merge(t1, t2)
8      else:
9          return t
10
11 def split(t):
12     """ precondition: len(t) >= 2 """
13     mid = len(t) // 2
14     t1 = t[:mid]
15     t2 = t[mid:]
16     return (t1, t2)
17
18 def merge(t1, t2):
19     if len(t1) == 0:
20         return t2
21     elif len(t2) == 0:
22         return t1
23     elif t1[-1] > t2[-1]:
24         last = t1.pop()
25         new = merge(t1, t2)
26         new.append(last)
27         return new
28     else:
29         last = t2.pop()
30         new = merge(t1, t2)
31         new.append(last)
32         return new

```

La complexité du tri par fusion est en temps $O(n \log_2 n)$, ce qui est bien meilleur que les tris par insertion et par sélection !

Le tri par fusion est en $O(n \log_2 n)$ – preuve

Nous allons analyser les 3 fonctions utilisées dans le tri par fusion et montrer que :

- `split` est en $O(n)$
- `merge` est en $O(n)$
- `merge_sort` est en $O(n \log_2 n)$

Notations : dans ce qui suit, n représente la taille de la liste t , n_1 la taille de t_1 et n_2 celle de t_2 .

$\implies$ notez que $n = n_1 + n_2$

Le tri par fusion est en $O(n \log_2 n)$ – preuve

```

11 def split(t):
12     """ precondition: len(t) >= 2 """
13     mid = len(t) // 2
14     t1 = t[:mid]
15     t2 = t[mid:]
16     return (t1, t2)

```

Analyse de la fonction `split`

- L'opérateur "slice" a une complexité $O(k)$ où k est la taille de la tranche retournée
- L'instruction 14 est donc en $O(n_1)$ et l'instruction 15 est en $O(n_2)$
- Les autres instructions sont en temps constant
- Le temps total de la fonction `split` est donc $O(n_1 + n_2) = O(n)$ $\square$

Le tri par fusion est en $O(n \log_2 n)$ – preuve

```

18 def merge(t1, t2):
19     if len(t1) == 0:
20         return t2
21     elif len(t2) == 0:
22         return t1
23     elif t1[-1] > t2[-1]:
24         last = t1.pop()
25         new = merge(t1, t2)
26         new.append(last)
27         return new
28     else:
29         last = t2.pop()
30         new = merge(t1, t2)
31         new.append(last)
32         return new

```

Analyse de la fonction merge

- Si $n_1 = 0$, on retourne (une référence vers) la liste t_2 , ce qui se fait en temps constant $O(1)$. Le cas $n_2 = 0$ est symétrique.
- Donc, si une des listes t_1 ou t_2 est vide $\Rightarrow O(1)$

Le tri par fusion est en $O(n \log_2 n)$ – preuve

```

18 def merge(t1, t2):
19     if len(t1) == 0:
20         return t2
21     elif len(t2) == 0:
22         return t1
23     elif t1[-1] > t2[-1]:
24         last = t1.pop()
25         new = merge(t1, t2)
26         new.append(last)
27         return new
28     else:
29         last = t2.pop()
30         new = merge(t1, t2)
31         new.append(last)
32         return new

```

Analyse de la fonction merge

- Donc, si une des listes t_1 ou t_2 est vide $\Rightarrow O(1)$
- Evaluons $T(n)$ pour merge où $n = n_1 + n_2$
 - *Base.* Si $n = 1$, alors une des deux listes (t_1 ou t_2) est vide et donc $T(1) = O(1)$

Le tri par fusion est en $O(n \log_2 n)$ – preuve

Analyse de la fonction `merge`

```

18 def merge(t1, t2):
19     if len(t1) == 0:
20         return t2
21     elif len(t2) == 0:
22         return t1
23     elif t1[-1] > t2[-1]:
24         last = t1.pop()
25         new = merge(t1, t2)
26         new.append(last)
27         return new
28     else:
29         last = t2.pop()
30         new = merge(t1, t2)
31         new.append(last)
32         return new

```

- Donc, si une des listes t_1 ou t_2 est vide $\Rightarrow O(1)$
- Evaluons $T(n)$ pour `merge` où $n = n_1 + n_2$
 - ▶ *Base.* $T(1) = O(1)$
 - ▶ *Gén.* Si $n > 1$ et qu'une des deux listes est vide, alors $T(n) = O(1)$. Si aucune des deux listes n'est vide, considérons le premier cas ($t_1[-1] > t_2[-1]$) (le second cas est symétrique). La condition est en $O(1)$ (instr. 23). Le coût du corps de cette branche (instr. 24 à 27), hors appel récursif, est en $O(1)$ car toutes les instructions sont en temps constant (travail en fin de liste). Lors de l'appel récursif, t_1 a un élément de moins. Donc, $T(n) = O(1) + T(n-1)$.

Le tri par fusion est en $O(n \log_2 n)$ – preuve

```

18 def merge(t1, t2):
19     if len(t1) == 0:
20         return t2
21     elif len(t2) == 0:
22         return t1
23     elif t1[-1] > t2[-1]:
24         last = t1.pop()
25         new = merge(t1, t2)
26         new.append(last)
27         return new
28     else:
29         last = t2.pop()
30         new = merge(t1, t2)
31         new.append(last)
32         return new

```

Analyse de la fonction merge

- Donc, si une des listes t_1 ou t_2 est vide $\Rightarrow O(1)$
- Evaluons $T(n)$ pour merge où $n = n_1 + n_2$
 - ▶ *Base.* $T(1) = O(1)$
 - ▶ *Gén.* Si $n > 1$, $T(n) = O(1) + T(n-1)$
 - ▶ La forme inductive de $T(n)$ est donc **identique** à celle vue pour `sum_rec` ci-avant et on peut en conclure que merge est aussi en $O(n)$ $\square$

Le tri par fusion est en $O(n \log_2 n)$ – preuve

```

1  def merge_sort(t):
2      n = len(t)
3      if n > 1:
4          (t1, t2) = split(t)
5          t1 = merge_sort(t1)
6          t2 = merge_sort(t2)
7          return merge(t1, t2)
8      else:
9          return t

```

Analyse de la fonction `merge_sort`

- *Base.* Si $n = 1$, alors $T(1) = O(1)$.
- *Gén.* Si $n > 1$, l'instr. 4 est en $O(n)$ (appel à `split`) et l'instr. 7 également (appel à `merge`). Les appels récurifs se font sur deux moitiés de t . Pour simplifier, on suppose que n est une puissance de 2. On obtient donc :

$$T(n) = O(n) + 2 T\left(\frac{n}{2}\right),$$

où n est une puissance de 2 (> 1)

Le tri par fusion est en $O(n \log_2 n)$ – preuve

Remarques :

- Supposer que n est une puissance de 2 est pratique car les listes seront toujours séparées en deux parties égales ($\frac{n}{2}$), jusqu'à tomber dans un cas de base (liste de longueur 1)
- Comme $T(n)$ ne décroît sûrement pas quand n croît, le fait d'étudier la croissance de $T(n)$ pour les puissances de 2 est suffisant. En effet, le fait de sauter certaines valeurs de $T(n)$ ne change pas sa croissance.

Le tri par fusion est en $O(n \log_2 n)$ – preuve

Avec les constantes, cela donne :

$$\begin{aligned} T(1) &= a, \\ T(n) &= bn + 2 T\left(\frac{n}{2}\right), \quad n > 1. \end{aligned}$$

Les premières valeurs sont :

$$\begin{aligned} T(1) &= a, \\ T(2) &= 2b + 2 T(1) = 2a + 2b, \\ T(4) &= 4b + 2 T(2) = 4a + 8b, \\ T(8) &= 8b + 2 T(4) = 8a + 24b, \\ T(16) &= 16b + 2 T(8) = 16a + 64b. \end{aligned}$$

Le coefficient de a semble être n . Analysons les coefficients de b :

n	2	4	8	16
Coef. de b	2	8	24	64

Le tri par fusion est en $O(n \log_2 n)$ – preuve

n	2	4	8	16
Coef. de b	2	8	24	64
Ratio Coef./ $n = \log_2 n$	1	2	3	4

Il apparaît que le ratio est $\log_2 n$ et donc que le coefficient de b est $n \log_2 n$. D'une manière générale, on peut prouver par récurrence (exercice) que si n est une puissance de 2 telle que

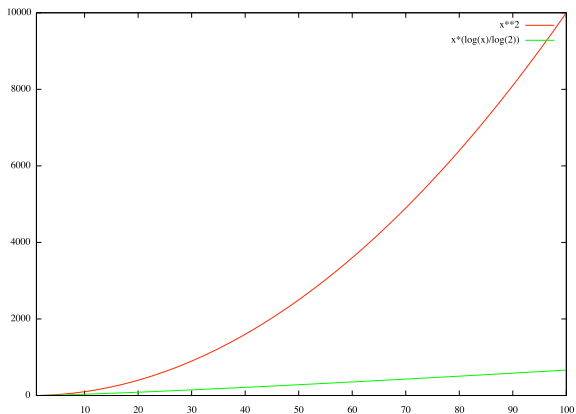
$$\begin{aligned} T(1) &= a, \\ T(n) &= bn + 2 T\left(\frac{n}{2}\right), \quad n > 1, \end{aligned}$$

alors $T(n) = an + bn \log_2 n$. On en conclut que $T(n) \in O(n \log_2 n)$ □

Analyse du comportement de nos algorithmes

Comparaison de la complexité des tris

Le tri par fusion est en $O(n \log_2 n)$ et les deux autres algorithmes de tris sont en $O(n^2)$.



Comparaison de la complexité des tris

Il existe beaucoup d'autres algorithmes de tris, joliment illustrés sur le site :

<https://www.toptal.com/developers/sorting-algorithms>.

La meilleure complexité possible pour un tri est en $O(n \log n)$ (par ex., tri par fusion et quicksort).

Comparaison des algorithmes de tri

Comparaison des trois fonctions de tri (t_1 déjà trié, t_2 trié par ordre décroissant, t_3 aléatoire) en fonction de la taille n de la séquence :

Temps affichés en msec

n	t1: sel	ins	mer	t2: sel	ins	mer	t3: sel	ins	mer
100	0.96	0.05	0.71	0.97	1.79	0.64	0.94	0.95	0.89
200	3.40	0.10	1.52	3.46	6.93	1.37	3.36	3.44	1.97
300	7.64	0.15	2.37	8.02	15.46	2.13	7.52	7.43	3.15
400	13.41	0.20	3.26	13.80	27.59	3.02	13.37	14.08	4.38
500	21.77	0.30	4.55	21.85	42.71	3.95	20.82	22.43	5.72
600	30.39	0.30	5.21	31.13	62.16	4.64	30.34	32.18	7.19
700	41.54	0.35	6.06	42.85	87.28	5.46	41.03	45.52	8.48
800	53.88	0.40	7.10	56.05	118.52	6.33	54.20	57.36	9.92
900	68.56	0.45	7.88	71.28	142.15	7.40	68.54	71.84	11.37
1000	84.56	0.50	8.77	87.87	177.36	8.46	85.01	90.58	12.05

Ces expérimentations confirment notre analyse grand- $\mathcal{O}$. En ce qui concerne les temps pour les listes déjà triées : voir la fin du chapitre.

Comparaison de la complexité du calcul de l'exposant

Au chapitre 8, nous avons introduit le calcul d'une exponentielle dite rapide. Mais quelle est réellement sa complexité ?

```

1  def expo(a, n):
2      r = 1
3      for i in range(n):
4          r = a * r
5      return r

```

```

1  def expo(a, n):
2      r = 1
3      while n > 0:
4          if n % 2 == 0:
5              a = a * a
6              n = n // 2
7          else:
8              r = r * a
9              n = n - 1
10     return r

```

La version simple calcule a^n en $O(n)$ (preuve laissée en exercice) et la version améliorée en $O(\log_2 n)$.

La version améliorée du calcul de l'exposant est en $O(\log_2 n)$ – preuve

```

1  def expo(a, n):
2      r = 1
3      while n > 0:
4          if n % 2 == 0:
5              a = a * a
6              n = n // 2
7          else:
8              r = r * a
9              n = n - 1
10     return r

```

- il est clair que le cas pair diminue n plus vite
- si n était une puissance de 2, c'est à dire $n = 2^p$ où $p \geq 1$
 - ▶ on tomberait toujours dans le cas pair jusqu'à obtenir $n = 1$
 - ▶ il y aurait exactement $p + 1$ itérations, où $p = \log_2 n$ (meilleur des cas)
- Imaginons maintenant le pire des cas : n est impair, et chaque fois que l'on divise n par 2, on retombe dans le cas impair
 - ▶ dans ce cas, le nombre d'itérations peut être doublé par rapport au meilleur cas
 - ▶ on conclut que le temps d'exécution dans le pire des cas est $O(2 \log_2 n) = O(\log_2 n)$ $\square$

Comparaison des algorithmes du calcul de l'exposant

Comparaison des deux fonctions pour le calcul de l'exposant :

```
Temps affichés en msec. Calcul de 2^n
n:  class: amel:
1000 0.112 0.012
2000 0.266 0.014
3000 0.536 0.019
4000 0.776 0.024
5000 1.294 0.025
6000 1.654 0.031
7000 2.121 0.027
8000 3.036 0.037
9000 2.992 0.039
10000 3.636 0.051
```

La notion de complexité explique ces résultats.

Comparaison de la complexité du calcul de Fibonacci

Il nous reste à comprendre pourquoi la version récursive du calcul de Fibonacci est si lente.

```

1  def fib_rec(n):
2      if n == 0:
3          return 0
4      elif n == 1:
5          return 1
6      else:
7          return fib_rec(n-1) + fib_rec(n-2)

```

```

1  def fib_iter(n):
2      f = [0, 1]
3      for i in range(2, n+1):
4          f.append(f[i-1] + f[i-2])
5      return f[n]

```

La version récursive calcule f_n en $O(\varphi^n)$ et la version itérative en $O(n)$ ($\varphi = \frac{1+\sqrt{5}}{2} \simeq 1.618$ est le nombre d'or).

La version itérative de Fibonacci est en $O(n)$ – preuve

```

1  def fib_iter(n):
2      f = [0, 1]
3      for i in range(2, n+1):
4          f.append(f[i-1] + f[i-2])
5      return f[n]

```

- les instructions 2 et 5 sont en $O(1)$
- grâce à `append`, l'instruction 4 (et donc le corps de la boucle) est en $O(1)$
- la séquence créée par `range` et le nombre d'itérations sont en $O(n)$
- au total : $O(1) + O(n) + O(n) \times O(1) = O(n)$ $\square$

La version récursive de Fibonacci est en $O(1.618^n)$ – preuve intuitive

```

1  def fib_rec(n):
2      if n == 0:
3          return 0
4      elif n == 1:
5          return 1
6      else:
7          return fib_rec(n-1) + fib_rec(n-2)

```

Pas une preuve formelle mais explique intuitivement le résultat :

- Clairement $T(n) = T(n-1) + T(n-2)$
- on peut penser que chaque appel récursif “double” approximativement le nombre d’appels
 - ▶ On aurait alors une fonction du type $T(n) = 2^n$
 - ▶ Mais cela ne marche pas car $2^n \neq 2^{n-1} + 2^{n-2}$

La version récursive de Fibonacci est en $O(1.618^n)$ – preuve intuitive

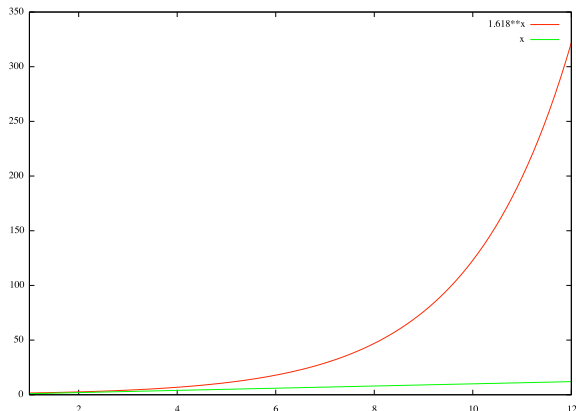
- Gardons l'idée d'une exponentielle $T(n) = a^n$ et essayons de déterminer sa base.
- Comme $T(n) = T(n-1) + T(n-2)$, il s'agit de résoudre

$$a^n = a^{n-1} + a^{n-2}$$

- En divisant les deux membres par a^{n-2} on obtient $a^2 = a + 1$
- Les racines de cette équation sont $\frac{1+\sqrt{5}}{2} \simeq 1.618$ et $\frac{1-\sqrt{5}}{2} \simeq -0.618$
- Comme les fonctions de complexité doivent retourner quelque chose de positif (un temps), on ne garde que la première solution, qui est le nombre d'or. $\square$

Comparaison de la complexité du calcul de Fibonacci

La complexité du calcul récursif de Fibonacci est donc exponentielle !



Compléments à lire par soi-même

Evaluer le temps CPU d'un algorithme

Pour estimer le temps CPU en Python, on utilise le module `timeit` qui règle automatiquement la plupart des problèmes évoqués au début du chapitre.

Néanmoins, cela reste une estimation, et le même morceau de code sur les mêmes entrées peut donner des temps différents.

C'est pourquoi il est recommandé de :

- lancer plusieurs séries de tests ;
- pour chaque série, calculer la moyenne des temps CPU ;
- ne garder que la **plus petite** de ces moyennes.

Voyez-vous pourquoi on prend la plus petite moyenne ?

Evaluer le temps CPU d'un algorithme

Pour estimer le temps CPU en Python, on utilise le module `timeit` qui règle automatiquement la plupart des problèmes évoqués au début du chapitre.

Néanmoins, cela reste une estimation, et le même morceau de code sur les mêmes entrées peut donner des temps différents.

C'est pourquoi il est recommandé de :

- lancer plusieurs séries de tests ;
- pour chaque série, calculer la moyenne des temps CPU ;
- ne garder que la **plus petite** de ces moyennes.

Voyez-vous pourquoi on prend la plus petite moyenne ?

Car les différences de temps ne sont pas dues à votre code (s'il est déterministe), mais à d'autres facteurs (e.g., antivirus, client mail, ou tout autre programme qui tourne en fond...) !

Utilisation du module `umons_cpu`

Pour faciliter et automatiser l'utilisation de `timeit`, nous avons écrit un module `umons_cpu` qui contient deux fonctions utiles :

- `cpu_time(f, *args)` pour calculer “comme il se doit” le temps CPU d'une fonction `f` ;
- `calibrate(f, *args)` pour connaître le nombre de tests réalisés par `cputime` pour déterminer le temps CPU de `f`.

Le module `umons_cpu` et un script `cpu_examples` sont disponibles sur moodle.

Utilisation du module `umons_cpu`

```
>>> import umons_cpu
>>> help(umons_cpu.cpu_time)
Help on function cpu_time in module umons_cpu:
```

```
cpu_time(f, *args)
    Retourne un temps CPU exprimé en millisecondes (ms)
    - f : fonction ou méthode à tester
    - *args : liste d'arguments pour f. Ces arguments ne sont pas
      modifiés, même si la fonction f a des effets de bord (ils sont
      copiés avant l'exécution).
```

Exemples :

```
    cputime(math.sqrt, 4)
        pour calculer le temps CPU de math.sqrt(4)
    cputime(str.upper, 'hello')
        pour calculer le temps CPU de 'hello'.upper()
    cputime(myfunc, x, y, z)
        pour calculer le temps CPU de myfunc(x, y, z)
```

Utilisation du module `umons_cpu`

Petite explication sur le fonctionnement de la fonction `cpu_time` :

- elle réalise 3 séries de n tests ;
- n est une puissance de 10 déterminée automatiquement : elle vaut 10 au minimum, mais sera plus grande si les premiers tests sont rapides ;
- seule la plus rapide des 3 séries est utilisée ;
- le temps CPU retourné est le temps moyen d'un test, calculé pour la plus rapide des 3 séries.

Utilisation du module `umons_cpu`

```
>>> import umons_cpu
>>> help(umons_cpu.calibrate)
Help on function calibrate in module umons_cpu:
```

```
calibrate(f, *args)
```

Retourne un nombre de tests qui rend le calcul du temps CPU
a priori raisonnable.

- `f` : fonction ou méthode à tester
- `*args` : liste d'arguments pour `f`. Ces arguments ne sont pas modifiés, même si la fonction `f` a des effets de bord (ils sont copiés avant l'exécution).

Le nombre de tests retourné est une puissance de 10 (au minimum 10). Il sera d'autant plus grand si la fonction semble rapide.

Utilisation du module `umons_cpu`

```
>>> from umons_cpu import calibrate
>>> from fibo import fib_iter, fib_rec
>>> calibrate(fib_iter, 30)
10000
>>> calibrate(fib_rec, 30)
10
```

Utilisation du module `umons_cpu`

La fonction `cpu_time` réalise une copie en profondeur des arguments de la fonction à tester, pour éviter les effets de bord. Cette copie prend un certain temps qui est inclus dans le temps CPU retourné. Pour beaucoup d'algorithmes, ce temps est négligeable.

Si la fonction à tester n'a pas d'effet de bord et que son temps d'exécution est très rapide, et si les arguments de la fonction sont lourds à copier, il peut-être utile d'utiliser plutôt `cpu_time_without_copy`.

Formatage des tableaux de temps CPU

A la fin du module fibo :

```
if __name__ == '__main__':
    from umons_cpu import cpu_time
    print("Temps affichés en msec")
    print('n:      iter:  rec:')
    for i in range (5, 31, 5):
        print('%4d ' % i, end=' ')
        print('%3f ' % (cpu_time(fib_iter, i)), end=' ')
        print('%3f' % (cpu_time(fib_rec, i)))
    for i in range (100, 1901, 200):
        print('%4d ' % i, end=' ')
        print('%3f ----' % (cpu_time(fib_iter, i)))
```

Formatage des tableaux de temps CPU

Dans le module `sort` :

```

from random import randint
from umons_cpu import cpu_time

def test(n):
    t1 = list(range(n))
    t2 = list(range(n,0,-1))
    t3 = []
    for i in range(n):
        t3.append(randint(0,n))
    print('%6d ' % n, end=' ')
    print('%7.2f' % (cpu_time(selection_sort, t1)), end=' ')
    print('%7.2f' % (cpu_time(insertion_sort, t1)), end=' ')
    print('%7.2f' % (cpu_time(merge_sort, t1)), end=' ')
    print('    %7.2f' % (cpu_time(selection_sort, t2)), end=' ')
    print('%7.2f' % (cpu_time(insertion_sort, t2)), end=' ')
    print('%7.2f' % (cpu_time(merge_sort, t2)), end=' ')
    print('    %7.2f' % (cpu_time(selection_sort, t3)), end=' ')
    print('%7.2f' % (cpu_time(insertion_sort, t3)), end=' ')
    print('%7.2f' % (cpu_time(merge_sort, t3)))

if __name__ == '__main__':
    print("Temps affichés en msec")
    print('n      t1: sel      ins      mer      t2: sel      ins      mer      t3: sel      ins      mer')
    for i in range(100, 901, 100):
        test(i)

```

Preuves

$$f(n) \in O(g(n)) \iff \exists n_0 \in \mathbb{N}, c \in \mathbb{R}^+ \text{ t.q. } \forall n \geq n_0 : f(n) \leq c g(n)$$

Exemples de preuves :

- Soit $f(n) = 2n^2$ et $g(n) = n^2$, alors $f(n) \in O(g(n))$. En effet, si $n_0 = 0$ et $c = 2$, alors $f(n) \leq 2g(n)$, $\forall n \geq 0$.
- Soit $f(n) = (n+1)^2$ et $g(n) = n^2$, alors $f(n) \in O(g(n))$. En effet, si $n_0 = 1$ et $c = 4$, alors

$$f(n) = (n+1)^2 = n^2 + 2n + 1 \leq n^2 + 2n^2 + n^2 = 4n^2 = 4g(n), \quad \forall n \geq 1.$$

Notez que l'inégalité n'est pas vraie si $n = 0$, d'où $n_0 = 1$.

Remarque : on aurait pu prendre d'autres valeurs (comme $n_0 = 3$ et $c = 2$)

Preuves

- $n^2 \in O(n^3)$. Soit $n_0 = 0$ et $c = 1$: $n^2 \leq n^3, \forall n \geq 0$ est vrai (trivial).
- $n^3 \notin O(n^2)$. Par l'absurde, supposons que $n^3 \in O(n^2)$, alors il existe deux constantes n_0 et c telles que $n^3 \leq c n^2, \forall n \geq n_0$. Cela signifie que

$$c \geq \frac{n^3}{n^2} = n, \quad \forall n \geq \max(n_0, 1),$$

ce qui est une contradiction avec le fait que c est une constante (le $\max(n_0, 1)$ est là pour éviter une division par zéro si $n_0 = 0$).

Preuves

La notation grand- O permet de simplifier les fonctions de complexité.

Lemme (les facteurs constants ne sont pas importants)

Pour toute valeur constante $d > 0$ et toute fonction de complexité $f(n)$, on a

$$f(n) \in O(d \cdot f(n)) \text{ et } d \cdot f(n) \in O(f(n))$$

Preuve.

Soit $c = \frac{1}{d}$ et $n_0 = 0$, on a

$$f(n) = (c \cdot d)f(n) = c(d \cdot f(n)), \quad \forall n \geq 0,$$

et donc $f(n) \in O(d \cdot f(n))$. Pour le deuxième cas, il suffit de choisir $c = d$. □

Exemples : $1000n^2$ et $\frac{n^2}{1000} \in O(n^2)$.

Preuves

Lemme (les termes d'ordre inférieurs sont négligeables)

Soit une fonction de complexité $f(n)$ polynomiale et dont le coefficient du terme de plus grand degré est positif, c-à-d,

$$f(n) := a_k n^k + a_{k-1} n^{k-1} + \dots + a_2 n^2 + a_1 n + a_0,$$

où $a_k > 0$. Alors,

$$f(n) \in O(n^k)$$

Preuves

Preuve.

Soit $n_0 = 1$ et $c = \sum_{i=0}^k \max(a_i, 0)$ (somme des coefficients positifs).
Donc c est bien une constante strictement positive car $a_k > 0$. Alors,

$$f(n) \leq cn^k, \quad \forall n \geq 1.$$

En effet, pour chaque coefficient a_j :

- si $a_j \leq 0$, on a certainement $a_j n^j \leq 0, \forall n \geq 1$.
- si $a_j > 0$, on a $a_j n^j \leq a_j n^k, \forall n \geq 1$ car $j \leq k$.



Exemple : Soit $f(n) := 3n^5 + 10n^4 - 100n^3 + n + 1$. Alors
 $f(n) \in O(n^5)$. En effet, si $c = 3 + 10 + 1 + 1 = 15$, alors $f(n) \leq 15n^5$,
 $\forall n \geq 1$.

Preuves

Lemme (Transitivité des expressions grand- O)

Grand- O est une relation transitive, c-à-d que si $f \in O(g)$ et $g \in O(h)$, alors $f \in O(h)$.

Preuve.

Par définition,

$$f(n) \leq c_1 g(n), \quad \forall n \geq n_1,$$

$$g(n) \leq c_2 h(n), \quad \forall n \geq n_2.$$

Soit $n_0 = \max(n_1, n_2)$ et $c_0 = c_1 \cdot c_2$, alors,

$$f(n) \leq c_1 g(n) \leq c_1 c_2 h(n) = c_0 h(n), \quad \forall n \geq n_0.$$



Boucles `while` avec condition coûteuse

Exemple

```

1  def is_include(t1, t2):
2      i = 0
3      n1 = len(t1)
4      while i < len(t1) and t1[i] in t2:
5          i += 1
6      return i == n1

```

Cet algorithme retourne vrai ssi les éléments de t_1 sont tous présents dans t_2 . Le temps de cet algorithme est $O(n_1 \cdot n_2)$, où n_1 et n_2 sont les tailles de t_1 et t_2 .

Preuve.

Le temps des instr. 2, 3 et 6 sont en $O(1)$. La boucle est exécutée n_1 fois et son corps est en $O(1)$. Le temps de l'évaluation de la condition est en $O(n_2)$. Donc le temps de cet algorithme est $O(1) + n_1 \cdot (O(1) + O(n_2)) + O(n_2) = O(n_1 \cdot n_2)$. □

Tri par insertion : et si la liste est déjà triée ?

```

1  def insertion_sort(t):
2      n = len(t)
3      for i in range(1,n):
4          clef = t[i]
5          j = i - 1
6          while j >= 0 and t[j] > clef:
7              t[j+1] = t[j]
8              j = j - 1
9          t[j+1] = clef

```

- si la liste est triée de manière croissante, alors l'analyse de la boucle intérieure montre qu'elle sera exécutée en temps constant (le test $t[j] > \text{clef}$ étant faux pour chaque i)
- l'analyse complète donne alors un temps en $O(n)$ dans ce cas particulier, qui est en réalité le *meilleur* des cas
- cette analyse de la complexité explique les temps CPU observés

Tri par sélection : et si la liste est déjà triée ?

```
1  def selection_sort(t):
2      n = len(t)
3      for i in range(n-1):
4          small = i
5          for j in range(i+1, n):
6              if t[j] < t[small]:
7                  small = j
8          (t[i], t[small]) = (t[small], t[i])
```

Le temps du tri par sélection est insensible au fait que la liste soit triée ou non et reste donc en $O(n^2)$ (le nombre d'itérations des deux boucles sera identiques, quelque soit la manière dont les données sont ordonnées).

Glossaire

Glossaire

temps CPU : temps mis par le processeur pour exécuter un programme, le temps CPU varie d'une machine à l'autre et est difficile à estimer précisément.

instruction élémentaire : instruction qui est exécutée en temps constant.

notation grand- O : notation qui permet d'exprimer la croissance d'une fonction, de manière simplifiée.

complexité : la complexité d'un algorithme est la croissance, dans le pire des cas, de son temps d'exécution. Elle est exprimée grâce à la notation grand- O en fonction de la taille des entrées.